HW10-ICA-PartA_Adam_Nelson-Archer

April 9, 2026

1 HW10 Part A :: Object Detection with Python

COSC 6373 – Adam Nelson-Archer, 2140122

[12]:
```
!pip install -q tensorflow keras matplotlib numpy opencv-python Pillow
```

```
[notice] A new release of pip is available: 24.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

[13]:
```
import os
import struct
import urllib.request
import numpy as np
from numpy import expand_dims
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, Input, BatchNormalization,␣
↪LeakyReLU, ZeroPadding2D, UpSampling2D, concatenate
from tensorflow.keras.layers import add
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from matplotlib import pyplot
from matplotlib.patches import Rectangle
```

1.0.1 a. Download and save the pre-trained model weights.

[14]:
```
weights_url = "https://pjreddie.com/media/files/yolov3.weights"
weights_path = "yolov3.weights"

if not os.path.exists(weights_path):
    print("Downloading weights (237MB), this may take a moment...")
    req = urllib.request.Request(weights_url, headers={'User-Agent': 'Mozilla/5.
↪0'})
    with urllib.request.urlopen(req) as response, open(weights_path, 'wb') as␣
↪out_file:
        out_file.write(response.read())
    print("Downloaded weights!")
else:
```

```
    print("Weights already exist.")
```

```
Weights already exist.
```

1.0.2 b. Create the YOLOv3 model.

[15]:
```
def _conv_block(inp, convs, skip=True):
    x = inp
    count = 0
    for conv in convs:
        if count == (len(convs) - 2) and skip:
            skip_connection = x
        count += 1
        if conv['stride'] > 1: x = ZeroPadding2D(((1,0),(1,0)))(x)
        x = Conv2D(conv['filter'],
                   conv['kernel'],
                   strides=conv['stride'],
                   padding='valid' if conv['stride'] > 1 else 'same',
                   name='conv_' + str(conv['layer_idx']),
                   use_bias=False if conv['bnorm'] else True)(x)
        if conv['bnorm']: x = BatchNormalization(epsilon=0.001, name='bnorm_' +␣
 ↪str(conv['layer_idx']))(x)
        if conv['leaky']: x = LeakyReLU(alpha=0.1, name='leaky_' +␣
 ↪str(conv['layer_idx']))(x)
    return add([skip_connection, x]) if skip else x

def make_yolov3_model():
    input_image = Input(shape=(None, None, 3))
    # Layer  0 => 4
    x = _conv_block(input_image, [{'filter': 32, 'kernel': 3, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 0},
                                  {'filter': 64, 'kernel': 3, 'stride': 2,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 1},
                                  {'filter': 32, 'kernel': 1, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 2},
                                  {'filter': 64, 'kernel': 3, 'stride': 1,␣
 ↪'bnorm': True, 'leaky': True, 'layer_idx': 3}])
    # Layer  5 => 8
    x = _conv_block(x, [{'filter': 128, 'kernel': 3, 'stride': 2, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 5},
                        {'filter':  64, 'kernel': 1, 'stride': 1, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 6},
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 7}])
    # Layer  9 => 11
    x = _conv_block(x, [{'filter':  64, 'kernel': 1, 'stride': 1, 'bnorm':␣
 ↪True, 'leaky': True, 'layer_idx': 9},
```

```
                        {'filter': 128, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 10}])
  # Layer 12 => 15
  x = _conv_block(x, [{'filter': 256, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 12},
                        {'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 13},
                        {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 14}])
  # Layer 16 => 36
  for i in range(7):
      x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 16+i*3},
                            {'filter': 256, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 17+i*3}])
  skip_36 = x
  # Layer 37 => 40
  x = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 37},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 38},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 39}])
  # Layer 41 => 61
  for i in range(7):
      x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 41+i*3},
                            {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 42+i*3}])
  skip_61 = x
  # Layer 62 => 65
  x = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 2, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 62},
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 63},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 64}])
  # Layer 66 => 74
  for i in range(3):
      x = _conv_block(x, [{'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 66+i*3},
                            {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 67+i*3}])
  # Layer 75 => 79
  x = _conv_block(x, [{'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 75},
```

```
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 76},
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 77},
                        {'filter': 1024, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 78},
                        {'filter':  512, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 79}], skip=False)
  # Layer 80 => 82
  yolo_82 = _conv_block(x, [{'filter': 1024, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 80},
                             {'filter':  255, 'kernel': 1, 'stride': 1,␣
↪'bnorm': False, 'leaky': False, 'layer_idx': 81}], skip=False)
  # Layer 83 => 86
  x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 84}], skip=False)
  x = UpSampling2D(2)(x)
  x = concatenate([x, skip_61])
  # Layer 87 => 91
  x = _conv_block(x, [{'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 87},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 88},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 89},
                        {'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 90},
                        {'filter': 256, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True, 'layer_idx': 91}], skip=False)
  # Layer 92 => 94
  yolo_94 = _conv_block(x, [{'filter': 512, 'kernel': 3, 'stride': 1, 'bnorm':
↪ True,  'leaky': True,  'layer_idx': 92},
                             {'filter': 255, 'kernel': 1, 'stride': 1, 'bnorm':
↪ False, 'leaky': False, 'layer_idx': 93}], skip=False)
  # Layer 95 => 98
  x = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1, 'bnorm':␣
↪True, 'leaky': True,   'layer_idx': 96}], skip=False)
  x = UpSampling2D(2)(x)
  x = concatenate([x, skip_36])
  # Layer 99 => 106
  yolo_106 = _conv_block(x, [{'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 99},
                              {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 100},
                              {'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 101},
```

```
                                {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 102},
                                {'filter': 128, 'kernel': 1, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 103},
                                {'filter': 256, 'kernel': 3, 'stride': 1,␣
↪'bnorm': True,  'leaky': True,  'layer_idx': 104},
                                {'filter': 255, 'kernel': 1, 'stride': 1,␣
↪'bnorm': False, 'leaky': False, 'layer_idx': 105}], skip=False)
    model = Model(input_image, [yolo_82, yolo_94, yolo_106])
    return model

class WeightReader:
    def __init__(self, weight_file):
        with open(weight_file, 'rb') as w_f:
            major,    = struct.unpack('i', w_f.read(4))
            minor,    = struct.unpack('i', w_f.read(4))
            revision, = struct.unpack('i', w_f.read(4))
            if (major*10 + minor) >= 2 and major < 1000 and minor < 1000:
                w_f.read(8)
            else:
                w_f.read(4)
            transpose = (major > 1000) or (minor > 1000)
            binary = w_f.read()
        self.offset = 0
        self.all_weights = np.frombuffer(binary, dtype='float32')

    def read_bytes(self, size):
        self.offset = self.offset + size
        return self.all_weights[self.offset-size:self.offset]

    def load_weights(self, model):
        for i in range(106):
            try:
                conv_layer = model.get_layer('conv_' + str(i))
                if i not in [81, 93, 105]:
                    norm_layer = model.get_layer('bnorm_' + str(i))
                    size = np.prod(norm_layer.get_weights()[0].shape)
                    beta  = self.read_bytes(size) # bias
                    gamma = self.read_bytes(size) # scale
                    mean  = self.read_bytes(size) # mean
                    var   = self.read_bytes(size) # variance
                    weights = norm_layer.set_weights([gamma, beta, mean, var])
                if len(conv_layer.get_weights()) > 1:
                    bias   = self.read_bytes(np.prod(conv_layer.
↪get_weights()[1].shape))
                    kernel = self.read_bytes(np.prod(conv_layer.
↪get_weights()[0].shape))
```

```
                    kernel = kernel.reshape(list(reversed(conv_layer.
↪get_weights()[0].shape)))
                    kernel = kernel.transpose([2,3,1,0])
                    conv_layer.set_weights([kernel, bias])
                else:
                    kernel = self.read_bytes(np.prod(conv_layer.
↪get_weights()[0].shape))
                    kernel = kernel.reshape(list(reversed(conv_layer.
↪get_weights()[0].shape)))
                    kernel = kernel.transpose([2,3,1,0])
                    conv_layer.set_weights([kernel])
            except ValueError:
                pass
```

[16]:
```
# Define the model and load weights
model = make_yolov3_model()

# load the model weights
weight_reader = WeightReader('yolov3.weights')

# set the model weights into the model
weight_reader.load_weights(model)

print("YOLOv3 Model created and weights loaded.")
```

```
YOLOv3 Model created and weights loaded.
```

1.0.3 c. Make a prediction on three images of your choice and report the model's output.

[17]:
```
# Utility classes and functions for processing output
class BoundBox:
    def __init__(self, xmin, ymin, xmax, ymax, objness = None, classes = None):
        self.xmin = xmin
        self.ymin = ymin
        self.xmax = xmax
        self.ymax = ymax
        self.objness = objness
        self.classes = classes
        self.label = -1
        self.score = -1

    def get_label(self):
        if self.label == -1:
            self.label = np.argmax(self.classes)
        return self.label
```

```
    def get_score(self):
        if self.score == -1:
            self.score = self.classes[self.get_label()]
        return self.score

def _sigmoid(x):
    return 1. / (1. + np.exp(-x))

def decode_netout(netout, anchors, obj_thresh, net_h, net_w):
    grid_h, grid_w = netout.shape[:2]
    nb_box = 3
    netout = netout.reshape((grid_h, grid_w, nb_box, -1))
    nb_class = netout.shape[-1] - 5
    boxes = []
    netout[..., :2]  = _sigmoid(netout[..., :2])
    netout[..., 4:]  = _sigmoid(netout[..., 4:])
    netout[..., 5:]  = netout[..., 4][..., np.newaxis] * netout[..., 5:]
    netout[..., 5:] *= netout[..., 5:] > obj_thresh

    for i in range(grid_h*grid_w):
        row = i / grid_w
        col = i % grid_w
        for b in range(nb_box):
            objectness = netout[int(row)][int(col)][b][4]
            if(objectness.all() <= obj_thresh): continue
            x, y, w, h = netout[int(row)][int(col)][b][:4]
            x = (col + x) / grid_w
            y = (row + y) / grid_h
            w = anchors[2 * b + 0] * np.exp(w) / net_w
            h = anchors[2 * b + 1] * np.exp(h) / net_h
            classes = netout[int(row)][col][b][5:]
            box = BoundBox(x-w/2, y-h/2, x+w/2, y+h/2, objectness, classes)
            boxes.append(box)
    return boxes

def correct_yolo_boxes(boxes, image_h, image_w, net_h, net_w):
    new_w, new_h = net_w, net_h
    for i in range(len(boxes)):
        x_offset, x_scale = (net_w - new_w)/2./net_w, float(new_w)/net_w
        y_offset, y_scale = (net_h - new_h)/2./net_h, float(new_h)/net_h
        boxes[i].xmin = int((boxes[i].xmin - x_offset) / x_scale * image_w)
        boxes[i].xmax = int((boxes[i].xmax - x_offset) / x_scale * image_w)
        boxes[i].ymin = int((boxes[i].ymin - y_offset) / y_scale * image_h)
        boxes[i].ymax = int((boxes[i].ymax - y_offset) / y_scale * image_h)

def _interval_overlap(interval_a, interval_b):
    x1, x2 = interval_a
```

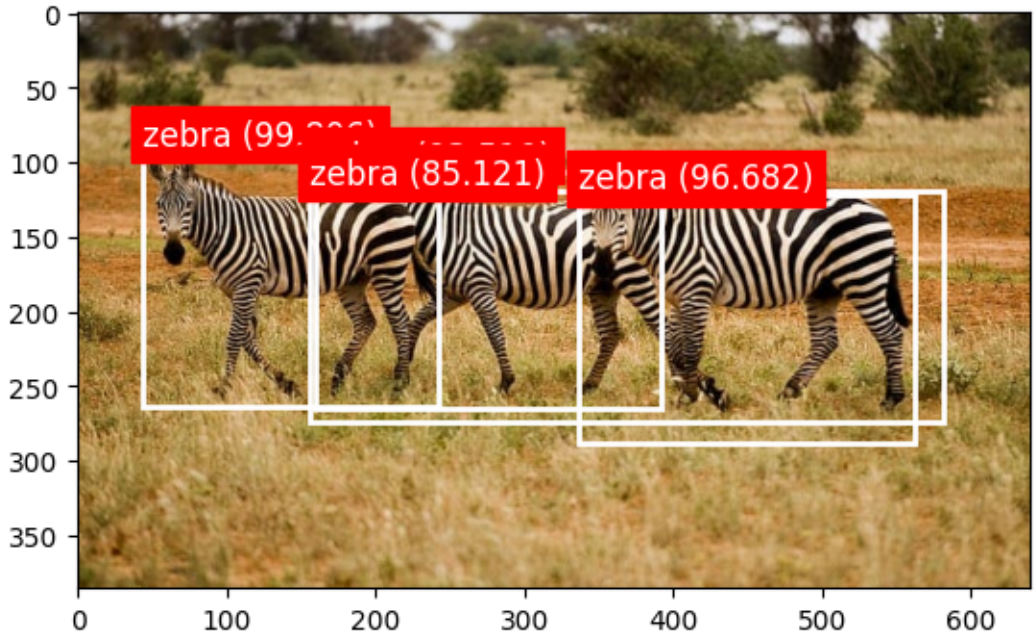
```
    x3, x4 = interval_b
    if x3 < x1:
        if x4 < x1: return 0
        else: return min(x2,x4) - x1
    else:
        if x2 < x3: return 0
        else: return min(x2,x4) - x3

def bbox_iou(box1, box2):
    intersect_w = _interval_overlap([box1.xmin, box1.xmax], [box2.xmin, box2.
 ↪xmax])
    intersect_h = _interval_overlap([box1.ymin, box1.ymax], [box2.ymin, box2.
 ↪ymax])
    intersect = intersect_w * intersect_h
    w1, h1 = box1.xmax-box1.xmin, box1.ymax-box1.ymin
    w2, h2 = box2.xmax-box2.xmin, box2.ymax-box2.ymin
    union = w1*h1 + w2*h2 - intersect
    return float(intersect) / union

def do_nms(boxes, nms_thresh):
    if len(boxes) > 0:
        nb_class = len(boxes[0].classes)
    else:
        return
    for c in range(nb_class):
        sorted_indices = np.argsort([-box.classes[c] for box in boxes])
        for i in range(len(sorted_indices)):
            index_i = sorted_indices[i]
            if boxes[index_i].classes[c] == 0: continue
            for j in range(i+1, len(sorted_indices)):
                index_j = sorted_indices[j]
                if bbox_iou(boxes[index_i], boxes[index_j]) >= nms_thresh:
                    boxes[index_j].classes[c] = 0

def load_image_pixels(filename, shape):
    image = load_img(filename)
    width, height = image.size
    image = load_img(filename, target_size=shape)
    image = img_to_array(image)
    image = image.astype('float32') / 255.0
    image = expand_dims(image, 0)
    return image, width, height

def get_boxes(boxes, labels, thresh):
    v_boxes, v_labels, v_scores = list(), list(), list()
    for box in boxes:
        for i in range(len(labels)):
```

```
            if box.classes[i] > thresh:
                v_boxes.append(box)
                v_labels.append(labels[i])
                v_scores.append(box.classes[i]*100)
    return v_boxes, v_labels, v_scores

def draw_boxes(filename, v_boxes, v_labels, v_scores):
    data = pyplot.imread(filename)
    pyplot.imshow(data)
    ax = pyplot.gca()
    for i in range(len(v_boxes)):
        box = v_boxes[i]
        y1, x1, y2, x2 = box.ymin, box.xmin, box.ymax, box.xmax
        width, height = x2 - x1, y2 - y1
        rect = Rectangle((x1, y1), width, height, fill=False, color='white',␣
 ↪linewidth=2)
        ax.add_patch(rect)
        label = "%s (%.3f)" % (v_labels[i], v_scores[i])
        pyplot.text(x1, y1-5, label, color='white', fontsize=12,␣
 ↪backgroundcolor='red')
    pyplot.show()
```

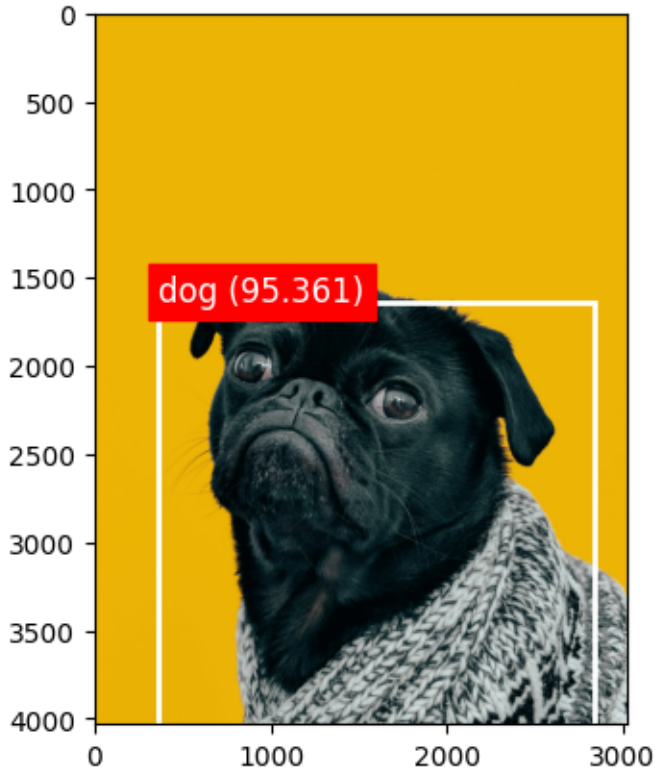
[18]:
```
# Prepare images
urls = [
    ("https://machinelearningmastery.com/wp-content/uploads/2019/03/zebra.jpg",␣
 ↪"image1.jpg"),
    ("https://images.unsplash.com/photo-1517849845537-4d257902454a", "image2.
 ↪jpg"), # Dog
    ("https://images.unsplash.com/photo-1514888286974-6c03e2ca1dba", "image3.
 ↪jpg")  # Cat
]

for url, name in urls:
    if not os.path.exists(name):
        req = urllib.request.Request(url, headers={'User-Agent': 'Mozilla/5.0'})
        with urllib.request.urlopen(req) as response, open(name, 'wb') as␣
 ↪out_file:
            out_file.write(response.read())

images = ["image1.jpg", "image2.jpg", "image3.jpg"]

# Configuration
input_w, input_h = 416, 416
anchors = [[116,90, 156,198, 373,326], [30,61, 62,45, 59,119], [10,13, 16,30,␣
 ↪33,23]]
```

```
class_threshold = 0.8  # Increased from article's 0.6 to 0.8 to filter out␣
↪false positives.
labels = ["person", "bicycle", "car", "motorbike", "aeroplane", "bus", "train",␣
↪"truck",
          "boat", "traffic light", "fire hydrant", "stop sign", "parking␣
↪meter", "bench",
          "bird", "cat", "dog", "horse", "sheep", "cow", "elephant", "bear",␣
↪"zebra", "giraffe",
          "backpack", "umbrella", "handbag", "tie", "suitcase", "frisbee",␣
↪"skis", "snowboard",
          "sports ball", "kite", "baseball bat", "baseball glove",␣
↪"skateboard", "surfboard",
          "tennis racket", "bottle", "wine glass", "cup", "fork", "knife",␣
↪"spoon", "bowl", "banana",
          "apple", "sandwich", "orange", "broccoli", "carrot", "hot dog",␣
↪"pizza", "donut", "cake",
          "chair", "sofa", "pottedplant", "bed", "diningtable", "toilet",␣
↪"tvmonitor", "laptop", "mouse",
          "remote", "keyboard", "cell phone", "microwave", "oven", "toaster",␣
↪"sink", "refrigerator",
          "book", "clock", "vase", "scissors", "teddy bear", "hair drier",␣
↪"toothbrush"]

def detect_and_draw(image_path, nms_threshold=0.5):
    image, image_w, image_h = load_image_pixels(image_path, (input_w, input_h))
    yhat = model.predict(image, verbose=0)
    boxes = list()
    for i in range(len(yhat)):
        boxes += decode_netout(yhat[i][0], anchors[i], class_threshold,␣
↪input_h, input_w)
    correct_yolo_boxes(boxes, image_h, image_w, input_h, input_w)
    do_nms(boxes, nms_threshold)
    v_boxes, v_labels, v_scores = get_boxes(boxes, labels, class_threshold)
    print(f"\n--- Detection on {image_path} (NMS={nms_threshold}) ---")
    for i in range(len(v_boxes)):
        print(f"Detected {v_labels[i]} with score {v_scores[i]:.2f}%")
    draw_boxes(image_path, v_boxes, v_labels, v_scores)

for img in images:
    detect_and_draw(img)
```

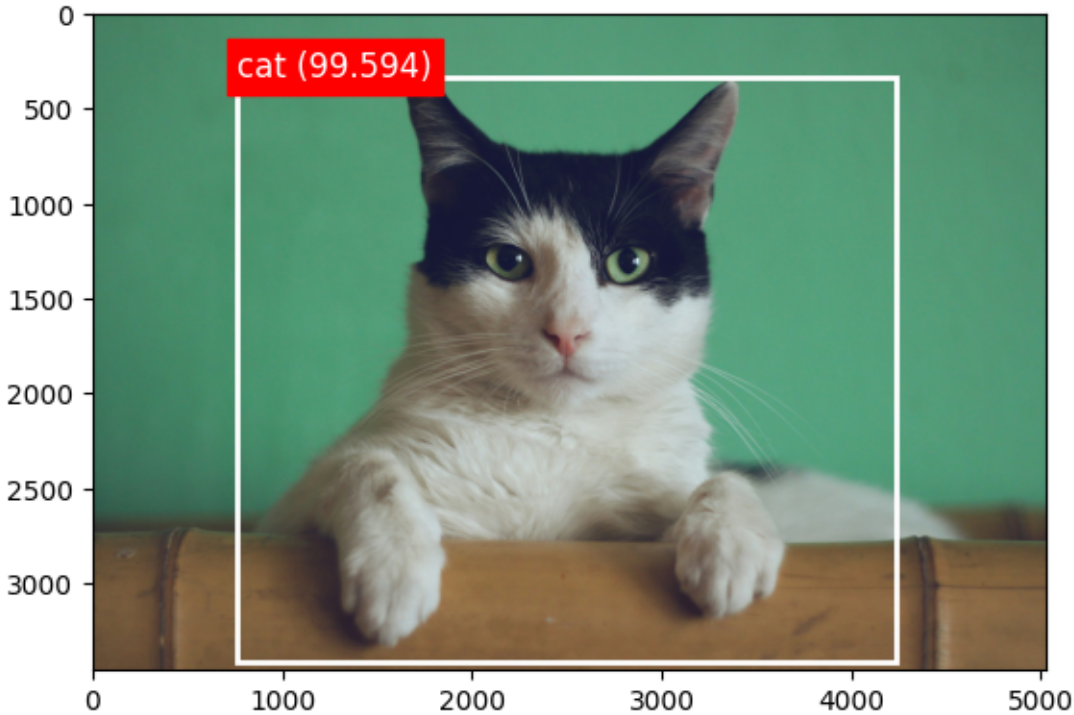
```
--- Detection on image1.jpg (NMS=0.5) ---
Detected zebra with score 99.81%
Detected zebra with score 93.51%
Detected zebra with score 85.12%
Detected zebra with score 96.68%
```




```
--- Detection on image2.jpg (NMS=0.5) ---
Detected dog with score 95.36%
```



```
--- Detection on image3.jpg (NMS=0.5) ---
Detected cat with score 99.59%
```



1.0.4 d. What is the purpose of the non-max suppression?

Non-maximum suppression (NMS) is a post-processing step used in object detection models like YOLO. The model often predicts multiple bounding boxes for the same object. NMS is used to filter out overlapping bounding boxes and retain only the one with the highest confidence score.

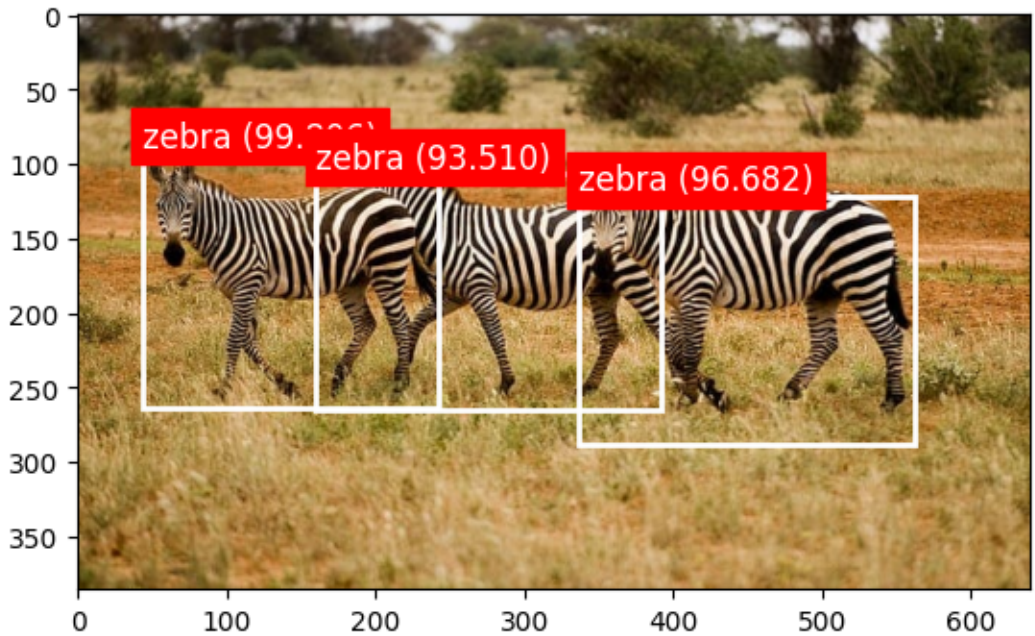
It calculates the Intersection over Union (IoU) of overlapping boxes and suppresses (removes) boxes that have an IoU greater than a predefined threshold, making sure that each object is detected only once.

1.0.5 e. You notice that the model's non-max suppression is 0.5. Please repeat step 3 using the following values for non-max suppression: (i) 0.3, and (ii) 0.8. What do you observe?

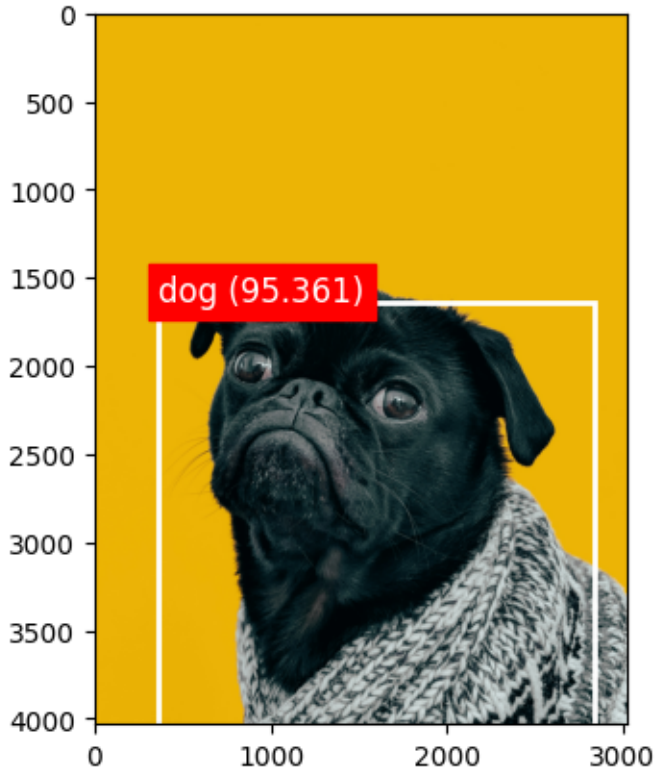
[19]:
```
print("Testing NMS = 0.3")
for img in images:
    detect_and_draw(img, nms_threshold=0.3)
```

```python
print("Testing NMS = 0.8")
for img in images:
    detect_and_draw(img, nms_threshold=0.8)
```

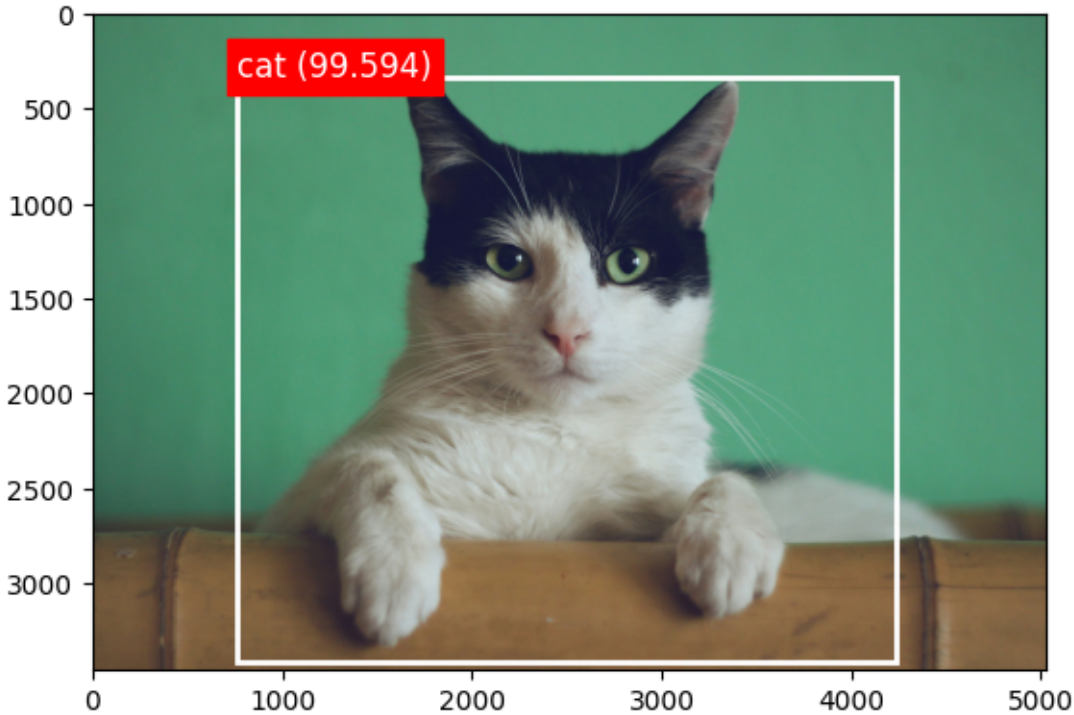
```
Testing NMS = 0.3

--- Detection on image1.jpg (NMS=0.3) ---
Detected zebra with score 99.81%
Detected zebra with score 93.51%
Detected zebra with score 96.68%
```



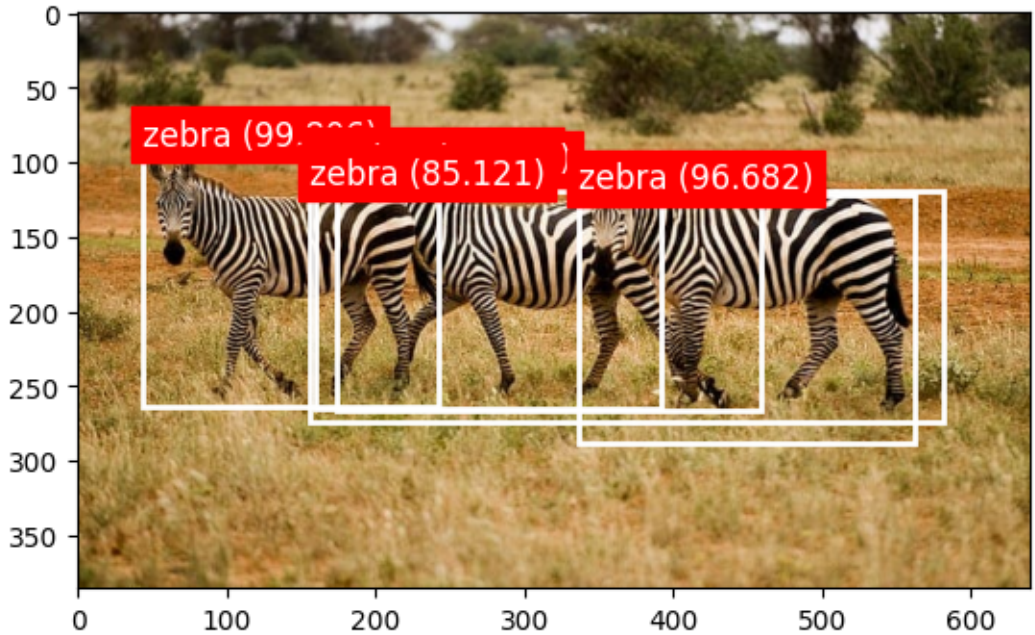

```
--- Detection on image2.jpg (NMS=0.3) ---
Detected dog with score 95.36%
```



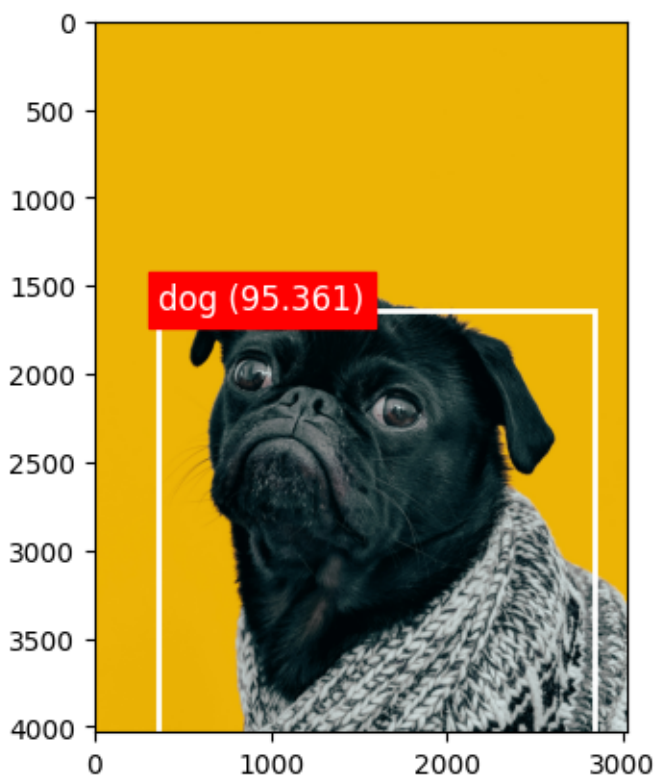

```
--- Detection on image3.jpg (NMS=0.3) ---
Detected cat with score 99.59%
```



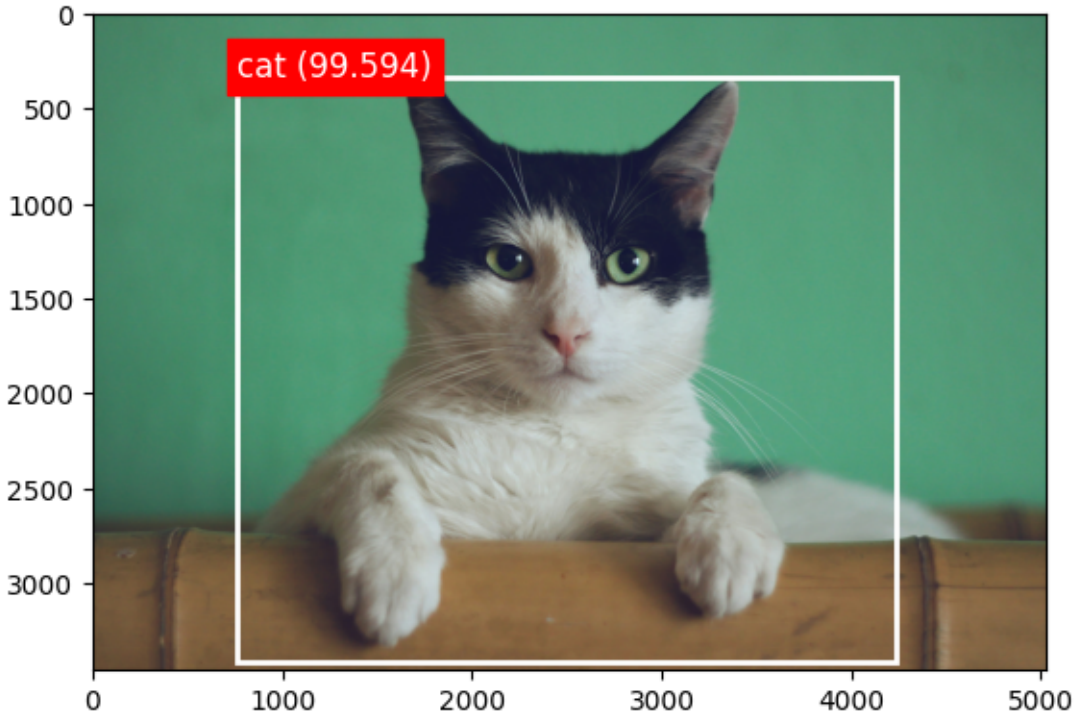

```
Testing NMS = 0.8

--- Detection on image1.jpg (NMS=0.8) ---
Detected zebra with score 99.81%
Detected zebra with score 93.51%
Detected zebra with score 89.15%
Detected zebra with score 85.12%
Detected zebra with score 96.68%
```




```
--- Detection on image2.jpg (NMS=0.8) ---
Detected dog with score 95.36%
```



```
--- Detection on image3.jpg (NMS=0.8) ---
Detected cat with score 99.59%
```



Observation: - **NMS = 0.3**: A lower threshold means suppression is more aggressive. Bounding boxes must overlap less to be removed. In some cases, this can lead to missed detections if objects are very close together or overlapping, but it generally removes redundant boxes effectively. - **NMS = 0.8**: A higher threshold means suppression is less aggressive. Boxes need to have a very high overlap (80%) to be suppressed. This often results in multiple bounding boxes being drawn for the same object because slightly offset boxes are not removed.

1.1 Acknowledgment

I used a GPT-5.3-Codex to help scaffold and organize this notebook.

Gemini-3.1 was used to check the result and validate conformity with the assignment outline.

All observations and responses were written by me, Adam Nelson-Archer.